

TECHNICAL & BEHAVIORAL INTERVIEW DOSSIER

TrackMySelf

20 Essential Interview Questions & High-Impact Model Answers

Production-Grade Mobile Engineering in Flutter, Dart & Drift SQLite v3

This document contains 20 fully articulated, interview-ready answers covering architectural decision-making, state management, database schema migrations, mathematical algorithms, hardware integration, automated test coverage, and live demonstration scripts.

Candidate / Developer:	Chethan G S
GitHub Repository:	github.com/Chethan4069/trackmyself
Primary Tech Stack:	Flutter 3.x, Dart 3.x, Drift ORM (SQLite v3), Provider, GoRouter
Automated Testing:	52 Unit / DAO / Domain Tests (100% Pass Rate)
Code Quality:	0 Lints / 0 Warnings (dart analyze clean)
Production Artifact:	Optimized Release APK (64.3 MB)

Designed for Mobile Software Engineer, Flutter Developer, and Full-Stack App Developer Interviews.

Interview Questions & Answers

QUESTION 1 Project Overview

Tell me about your project.

I built **TrackMySelf**, a production-grade, offline-first personal management and wellness mobile application developed using **Flutter, Dart, and Drift ORM (SQLite v3)**.

The application consolidates four fragmented aspects of daily personal life into a single cohesive, high-performance experience:

- **Health & Fitness:** Calorie tracking, macro breakdown, Mifflin-St Jeor metabolic calculations (BMR/TDEE), and visual body transformation progress galleries with weight delta tracking.
- **Daily Routines & Habits:** Habit tracking with an automated contiguous streak calculation engine, time-of-day filtering, and scheduled local notifications.
- **Time & Agendas:** An interactive monthly calendar with category-coded multi-event dot indicators and scheduled agenda reminders.
- **Memory & Asset Tracker:** A purchase and warranty management tool with camera/gallery receipt capture and automated 7-day pre-expiry alerts.

The application follows Clean Architecture with Provider state management, is validated by **52 automated unit, domain, and DAO tests (100% pass rate)**, has **0 static analysis warnings**, and compiles to an optimized 64 MB release APK.

QUESTION 2 Problem Statement & Value

What problem does your project solve?

Most users currently need 4 to 5 separate apps—one for calorie tracking (like MyFitnessPal), one for habits (like Habitica), one for calendar events, and another to keep track of purchase receipts and warranties. This creates app fatigue, high recurring subscription costs, and major privacy concerns because sensitive health and financial data are stored on third-party cloud servers.

TrackMySelf solves this by:

- **Consolidating Workflows:** Unifying fitness, habit building, agenda management, and asset tracking into a single executive dashboard with real-time vitals and summary tiles.
- **100% Data Privacy & Offline Guarantee:** Storing all relational data, photos, and calculations entirely on the device using an embedded SQLite database, requiring zero internet connection and zero cloud subscriptions.
- **Zero Latency:** Immediate response times with no network loading spinners or offline sync conflicts.

QUESTION 3

Motivation & Objectives

Why did you choose this project?

I chose this project because I wanted to tackle complex, real-world mobile engineering challenges beyond basic CRUD apps. I wanted to demonstrate:

- **Relational Data Modeling:** Designing a normalized SQLite database with foreign keys, cascading rules, and multi-user isolation across 7 tables.
- **Algorithmic Complexity:** Writing robust mathematical algorithms for metabolic equations (Mifflin-St Jeor), contiguous habit streak counting with date rollover tolerance, and real-time warranty lifecycle calculations.
- **Hardware & OS Integration:** Managing local background alarms, device reboot listeners, camera/gallery file system sandboxing, and haptic feedback.
- **Production Readiness:** Writing an automated test suite of 52 unit tests, adhering to strict clean architecture, and achieving zero static analysis warnings.

QUESTION 4

Ownership & Responsibility

What was your role in the project?

I was the **lead full-lifecycle mobile developer**. I was responsible for:

- Architecting the application using Clean Architecture and the Feature-First directory structure.
- Designing the Drift SQLite v3 schema and managing database migrations across versions.
- Implementing domain logic, state management (Provider), and custom UI components.
- Integrating native Android notification channels and sandboxed file system image storage.
- Writing the comprehensive test suite of 52 unit/DAO tests and generating the release build.

QUESTION 5

Teamwork & Collaboration

Did you work individually or in a team?

I worked on this project **individually**.

Doing this end-to-end allowed me to master every phase of modern Flutter app engineering—from UI design and state architecture to low-level SQLite schema migrations, hardware permission handling, test automation, and release APK optimization. It gave me a thorough, 360-degree understanding of mobile application development and lifecycle management.

QUESTION 6 Technology Stack & Decisions

What technologies did you use and why?

- **Flutter & Dart 3.x:** Cross-platform single codebase with native 60–120 FPS performance and strong compile-time type safety.
- **Drift ORM (SQLite):** Provides compile-time verified SQL queries, automatic schema migrations, and reactive `watch()` streams for live UI updates without runtime SQL syntax bugs.
- **Provider & ChangeNotifier:** Lightweight, decoupled reactive state management with low boilerplate and granular widget rebuild controls via `Consumer` and `Selector`.
- **GoRouter:** Declarative, URL-based routing supporting nested routes and authentication redirection guards.
- **flutter_local_notifications & flutter_timezone:** Reliable on-device background alarm scheduling without battery-draining foreground services.
- **image_picker & path_provider:** Capturing camera/gallery images and copying them from temporary cache into persistent sandboxed app documents storage.

QUESTION 7 System Architecture

Explain the architecture of your project.

The project follows **Clean Architecture** with a **Feature-First** structure.

Inside `lib/`, code is divided into `core/` (shared database, themes, notification services, shared widgets) and feature modules (`auth/`, `fitness/`, `habits/`, `calendar/`, `purchases/`, `home/`).

Each feature is organized into three distinct layers:

1. **Data Layer:** Contains DAOs (Data Access Objects) and database tables that communicate with Drift SQLite.
2. **Domain Layer:** Contains pure Dart entities, calculators (e.g., BMR, streaks, warranty state), and business rules with zero Flutter UI dependencies.
3. **Presentation Layer:** Contains UI screens, reusable widgets, and `ChangeNotifier` providers that bind the domain to the view.

This strict separation allowed me to test all business logic in isolation using in-memory databases.

QUESTION 8 Application Flow

Explain the complete flow of your application.

1. **App Launch & Auth Guard:** `main.dart` initializes timezones, spins up the Drift database singleton, and loads the active user session. If no session exists, `GoRouter` redirects to the Login/Registration screen.
2. **Onboarding & Profile Setup:** A new user inputs age, gender, height, weight, and activity level. The system calculates and persists their baseline BMR and TDEE.
3. **Executive Dashboard (Home):** The user sees a scenic sunrise mountain header, daily motivational quote, real-time vitals strip, and a 4-tile health card summarizing calories consumed vs. burned, active habits, and today's schedule.
4. **Daily Logging & Interaction:**
 - In **Fitness**, the user logs meals and workouts; progress rings update dynamically. Users can snap transformation progress photos and view weight deltas.
 - In **Habits**, checking off a habit triggers haptic feedback, increments the streak counter, and updates the daily completion banner.
 - In **Calendar**, the user schedules events with optional notification alerts.
 - In **Purchases**, the user logs items, attaches receipt photos, and sets warranty periods.

QUESTION 9 Technical Difficulty

What was the most difficult part of the project?

The most complex part was **retrofitting multi-user data isolation across legacy tables via Drift schema migrations (v1 to v3) without corrupting existing records**.

Initially, the schema was single-user. When introducing authentication, every table required a non-nullable `userId` foreign key. I had to write a custom `MigrationStrategy.onUpgrade` script that inspected schema versions, created the `Users` table, added the new foreign key column with fallback defaults to preserve legacy data, and ensured that every DAO query strictly filtered by `currentUserId` to prevent cross-account data leaks.

QUESTION 10 Engineering Challenges

What challenges did you face?

1. **Habit Streak Midnight Reset Bug:** Streaks were resetting to 0 if checked early in the morning before completing a habit for that day.
2. **Android 12+ / 14 Exact Alarm Restrictions:** Background reminders failed on newer Android versions due to strict battery optimization policies and uninitialized time zones.
3. **Image Cache & Disk Leaks:** Receipt and transformation photos were disappearing after OS cache cleanups, or lingering orphaned when records were deleted.
4. **UI Layout Constraints:** Images in the header and quote banners were either being clipped at the edges or distorted across different phone screen aspect ratios.

QUESTION 11 Problem Solving

How did you solve those challenges?

- **For Streaks:** I adjusted the streak anchor algorithm to accept either *today* OR *yesterday* as the valid contiguous starting date, providing a 1-day grace period.
- **For Notifications:** I initialized the tz database via `flutter_timezone`, declared `SCHEDULE_EXACT_ALARM` and `POST_NOTIFICATIONS` in `AndroidManifest.xml`, and registered `RECEIVE_BOOT_COMPLETED` to reschedule alarms after a reboot.
- **For Images:** I copied temporary camera files directly into `getApplicationDocumentsDirectory()` and implemented automatic `File(path).delete()` hooks when database records are deleted.
- **For UI Layout:** I wrapped images in responsive containers using `BoxFit.cover` paired with `Alignment.topCenter` and uncropped card structures.

QUESTION 12 Key Takeaways & Growth

What did you learn from this project?

- How to architect a true **offline-first** relational system in Flutter.
- Advanced SQLite schema migration strategies using Drift.
- How to write effective, fast-running unit tests using in-memory databases (`NativeDatabase.memory()`).
- Modern Android system permissions, background scheduling, and lifecycle management.
- How to optimize widget trees using `const` constructors, `TweenAnimationBuilder`, and selective rebuilding.

QUESTION 13 Future Roadmap

What would you improve if you had more time?

1. **Biometric Authentication:** Integrate `local_auth` for Face ID / Fingerprint unlock.
2. **Cloud Backup & Two-Way Sync:** Implement a sync service using Supabase or Firebase with timestamp vector conflict resolution.
3. **Data Export / Import:** Allow users to export and import their full history as an encrypted JSON or SQLite backup file.
4. **Wear OS / Smartwatch Companion:** A lightweight watch interface for quick habit check-offs and water logging.

QUESTION 14 Quality Assurance & Testing

How did you test your project?

I wrote **52 automated unit, DAO, and domain tests**:

- **Domain Unit Tests:** Validated pure mathematical functions (Mifflin-St Jeor formula, macro conversions, warranty state transitions).
- **DAO & Database Tests:** Spun up isolated in-memory SQLite instances (`NativeDatabase.memory()`) to verify CRUD operations, foreign-key cascade deletions, and query filters.
- **Edge Case Tests:** Tested leap years, 1-day grace periods in streaks, duplicate email registration, and 7-day pre-expiry alerts.
- **Static Analysis:** Enforced strict linting rules; `dart analyze` passes with 0 issues.

QUESTION 15 Error Handling & Reliability

How did you handle errors?

- **UI Input Validation:** Used `AutovalidateMode.onUserInteraction` with standard `FormState` keys to validate emails, positive numbers, and required fields in real time.
- **Async Try-Catch Blocks:** Wrapped database transactions and file I/O operations in structured `try-catch` blocks, surfacing user-friendly messages via `SnackBar` or modal alerts rather than raw technical errors.
- **Null Safety:** Strict compile-time Dart null-safety guarantees throughout the entire codebase.

QUESTION 16 Data Persistence

How did you store and retrieve data?

Data is persisted locally using **Drift ORM with SQLite v3**:

- **Structured Data:** Stored across 7 relational tables with indexes on `userId` and timestamp columns.
- **Retrieval:** For static screens, queries use `.get()`. For dynamic dashboards, queries use `.watch()`, which returns a reactive Dart `Stream` that automatically emits new data whenever an underlying table changes.
- **Binary Assets:** Receipt and progress images are saved to the phone's sandboxed document directory, with their file paths stored as strings in SQLite.

QUESTION 17 Application Security

How did you make your application secure?

1. **No Plaintext Passwords:** User passwords are encrypted/hashed with standard SHA-256 before being saved to SQLite.
2. **Multi-User Isolation:** Every query strictly enforces the authenticated `userId`, ensuring complete privacy on shared devices.
3. **Sandboxed File Storage:** All photos are kept inside the app's private sandboxed directory, inaccessible to other third-party apps on the device without root access.
4. **No External Exposure:** Because the app operates offline-first, personal data is never transmitted over insecure networks.

QUESTION 18 Scalability & Cloud Architecture

How would you scale this application for 10,000 to 100,000 users?

Since the app is offline-first, client performance is already decentralized—each phone runs its own SQLite engine independently, so client CPU and database costs do not multiply with user growth.

To add cloud synchronization for 100,000+ users:

1. **Backend:** Deploy a horizontally scalable REST or GraphQL API (Go, Node.js, or Supabase/PostgreSQL) behind an AWS Application Load Balancer.
2. **Sync Protocol:** Implement an incremental change-data-capture (CDC) sync engine using `updatedAt` timestamps and soft deletes (`isDeleted = true`) to sync only modified rows.
3. **Media Storage:** Offload receipt and body photos to Amazon S3 or Google Cloud Storage using pre-signed upload URLs and CDN caching (CloudFront).
4. **Caching & Queueing:** Use Redis for auth token caching and rate limiting.

QUESTION 19 Deployment & CI/CD

How would you deploy this project?

1. **Production Build:** Run `flutter build apk --release` (or `appbundle` for Google Play), using ProGuard/R8 to minify bytecode, strip unused resources, and tree-shake unused icons.
2. **Signing:** Configure `key.properties` with an RSA release keystore.
3. **CI/CD Pipeline:** Set up GitHub Actions or Codemagic to automatically run `flutter test` and `dart analyze` on every pull request, build the signed AAB/IPA, and deploy directly to Google Play Internal Testing / Apple TestFlight tracks.

QUESTION 20 Live Demonstration Script

Can you demonstrate the project?

"Absolutely! Let me walk you through the app on my device/emulator:

1. First, here is the **Login & Multi-User Switcher**. Let's log into our primary profile.
2. This brings us to the **Executive Home Dashboard**—notice the smooth scenic header, our daily quote, and the real-time calorie burn vs. intake ring.
3. Let's navigate to **Fitness**: notice how adding a meal instantly animates the macro bars. At the bottom, here is our **Body Transformation Gallery** comparing progress photos with weight delta badges.
4. In **Habits**, tapping to complete a routine triggers immediate haptic feedback and updates the contiguous streak.
5. Finally, in **Purchases**, you can see our dynamic warranty engine highlighting items that are expiring within 7 days, complete with full-screen receipt photo viewing.

Everything you see is updating via reactive SQLite streams with 100% offline persistence."